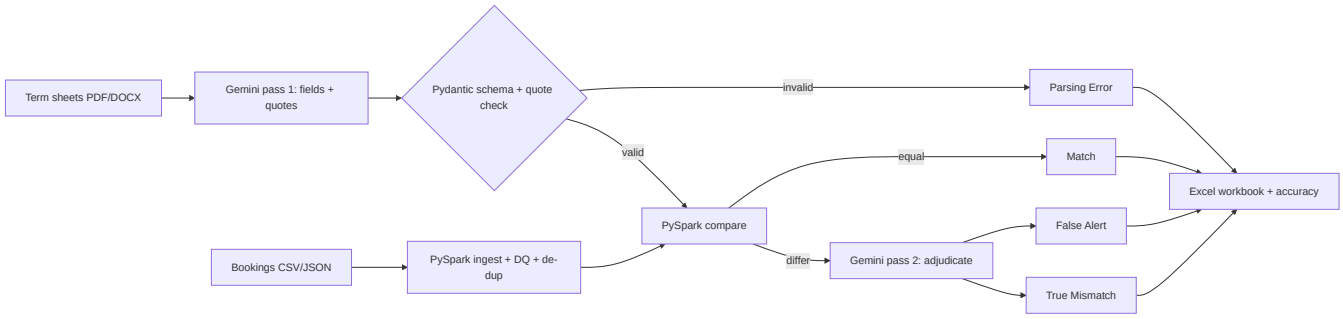
LLM-Powered Term Sheet Reconciliation: Solution Report



1. Workflow

1. **Parse.** PDF text comes from `pdfplumber`, with `pypdf` as a fallback. DOCX is read in body order, tables included.
2. **Extract (LLM pass 1).** One Gemini call per document returns 18 fields, each as `{value, quote, confidence, note}`. The call is schema-constrained via `response_json_schema`.
3. **Validate.** Each field is validated on its own against a strict Pydantic schema. A wrong type (`"9.25%"` for a float, `"14 October 2025"` for an ISO date, `"Rs."` for an ISO currency) becomes a **Parsing Error** and is never compared, so it can't appear as a mismatch. Every quote is checked against the normalised source text, which flags possible hallucinations.
4. **Ingest (PySpark).** CSV and JSON extracts are mapped to a canonical model. Dates (ISO, US, `14-Oct-2020`), numbers and lists are normalised. Malformed rows, unparseable values and missing columns are logged, and identical CSV/JSON copies collapse (20 rows → 10 trades).
5. **Compare (PySpark).** Trades are linked to their term sheet by ISIN, or by issuer when the term sheet has none, as with IDBI. Bookings are unpivoted and compared by type: text, numeric with tolerance, ISO date, perpetual, unordered list. There are no Python UDFs, so the code runs reliably on Windows. Two notional rules also run: the notional must be a whole multiple of the denomination, and a notional that differs from the dominant size booked in the same bond is flagged as an **Outlier (Review)**.
6. **Adjudicate (LLM pass 2).** Mismatches are de-duplicated (one ISIN-format break across 5 trades = 1 question). They're sent in one batch with the term-sheet **verbatim quote** and the booking value, and Gemini returns `TRUE_MISMATCH | FALSE_ALERT`, a reason and a confidence.
7. **Report.** An Excel workbook with Summary, Trade Summary (BREAK / REVIEW / CLEAN), Exceptions, Detail with evidence, Extraction audit, **Extraction Accuracy**, Bookings and Data Quality sheets.

2. LLM / API integration

- **Gemini Flash** (free tier, `gemini-3.8-flash`; a retired model automatically switches to Google's named replacement) via the official `google-genai` SDK. The key comes from `.env` and is never logged. Temperature is 0, and JSON output is constrained by schema. If the API rejects the schema, the client falls back to plain JSON mode automatically.
- **Resilience.** `tenacity` retries 408/429/5xx, timeouts and malformed JSON with exponential backoff and jitter. A client-side throttle respects the free-tier rate limit. Auth errors fail fast with a hint.
- **Never fail silently.** If extraction fails, the regex extractor fills in (labelled `heuristic-fallback`). If adjudication fails, deterministic equivalence rules decide, labelled `rules-fallback`. They only clear a break when the values are provably equivalent; anything uncertain stays a **True Mismatch** or is flagged for review.
- **Cost and reproducibility.** A full run takes 3 API calls. Responses are cached by prompt hash, so re-runs are free. `--record` saves a live run to `recordings/`, and `--provider replay` lets reviewers see the real Gemini output with **no key and no network**. A rule-based `mock` provider keeps the tests and CI fully offline.
- **Pluggable.** Providers implement a small `LLMClient` interface (OpenAI or Groq would be one class). Prompts are versioned (`PROMPT_VERSION`, shown on every report).

3. Field extraction design

- A single registry (`FIELD_SPECS`) maps each field to its booking column, comparison kind, label and LLM guidance. Adding a field is one line; the prompt, Gemini schema, comparison and report follow automatically.
- The prompt makes the model a **conservative analyst**: quote verbatim, return `null` with a reason rather than guess, convert units (crore/lakh, Indian digit grouping `10,00,000`), use ISO dates, return `PERPETUAL` for perpetual bonds, and explain derived values in `note` (e.g. first coupon = allotment + 1 year).
- **Measured, not assumed.** Hand-verified golden values (`data/expected`) score every extraction as Correct / Wrong / Missed / Hallucinated / Parsing error. In CI, the score gates changes to prompts or models (`--min-`

`accuracy 0.9`).

4. Assumptions

- Settlement date = issue / deemed allotment date when no separate date is given. "Expected to be" dates (Genel) are treated as contractual.
- Slash dates in bookings are US `M/d/yyyy` (the Genel CSV). In production this would be configured per source system.
- Issue amount means the base issue (IDBI: Rs.1,500 Cr, excluding the Rs.1,000 Cr green-shoe).
- A booked maturity on a perpetual bond is a True Mismatch. The 10-year call date (17 Oct 2024) doesn't explain 17 Oct 2025 either.
- Term sheets carry no trade notional, so the notional is checked by rules. The peer check is a review signal, not a break, because the term sheet can't prove a trade size. Minimum subscription is a primary-allocation rule and isn't applied per trade.
- Fields the term sheet doesn't state are reported as *Not in Term Sheet*, never silently counted as matches.

5. Challenges

- **Layout noise.** The IDBI PDF is a numbered two-column table with labels that wrap ("Deemed Date of / Allotment"). Sending the whole document to a long-context model avoids brittle chunking.
- **Units and conventions.** Crores/lakhs, percent vs decimal and synonyms (Bullet vs "repaid in full at maturity") are where false alerts come from. The second pass resolves them with an explanation instead of hard-coding every variant.
- **Hallucination.** This is mitigated by mandatory quotes checked against the source, strict schema validation, temperature 0, conservative defaults, and measured accuracy.
- **PySpark on Windows.** The pipeline uses no UDFs and no Spark file writes (so no `winutils`), pins the worker interpreter, and binds to localhost.

6. Production / DevOps suggestions

- **CI/CD.** GitHub Actions (included) runs lint (`ruff`), type checks (`mypy`), 43 offline tests, mock and replay end-to-end runs, and a live Gemini run with an accuracy gate when a repository secret is set. The Dockerfile gives a reproducible runtime (Python 3.11 + Java 17, non-root user).
- **Deployment.** Run it event-driven (a new term sheet lands in object storage) or on a schedule via Airflow or Databricks Workflows. Read bookings from Delta tables or the booking database, and write results to a recon table that feeds an ops dashboard and an exception workflow (ServiceNow / JIRA).
- **Scale.** Spark ingestion and comparison already scale horizontally. Extract each term sheet once per ISIN and reuse it across trades. LLM calls can use the Gemini Batch API or `mapInPandas` with a shared rate limiter.
- **Security.** Keep keys in a secrets manager (Vault / AWS Secrets Manager). Use an enterprise endpoint (Vertex AI: data residency, no training on data). Mask counterparty and PII data before prompting. Encrypt outputs and keep extraction JSON, quotes, prompt version and model as an audit trail.
- **Observability and governance.** Use structured logs and metrics: parsing-error rate, unverified-quote rate, false-alert ratio, accuracy drift, latency and cost. Route low-confidence or `needs_review` items to analysts, and feed their decisions back into the golden set. Add OCR (e.g. Document AI) for scanned term sheets, and instrument-specific schemas for FRNs, step-ups and multi-tranche documents.